SPLUNK — COMPLETE STUDY NOTES

DevOps Interview Preparation | Cognizant Training Program

PHASE 1 — THE MENTAL MODEL

What is Splunk?

Splunk is a centralized platform that pulls logs and machine data from every source in your organization — servers, cloud accounts, firewalls, applications, network devices — into one single place. Once the data is there, you can search it, analyze it, visualize it, and set up automatic alerts on it, all without logging into a single server.

In Azure terms, think of it like Azure Monitor and Log Analytics Workspace combined — but far more powerful and capable of ingesting data from any source, not just Azure resources.

The Problem Splunk Solves

WITHOUT SPLUNK	WITH SPLUNK
SSH into each server manually	All logs come to one place
Different log format per system	Normalized into searchable events
Manual grep through text files	SPL query returns results in seconds
No alerting until someone notices	Automatic real-time alerts
No historical trend visibility	Dashboards show patterns over time
Breach investigation takes days	Investigation takes minutes

The Splunk Data Pipeline

Everything Splunk does flows through exactly four stages. This is the most important mental model to carry into your interview.

Stage 1 — INPUT

Splunk receives the raw data stream. Breaks it into 64KB blocks and stamps metadata: host, source, sourcetype. Does NOT read the contents of the data at this stage.

Stage 2 — PARSING

Splunk examines the content. Creates individual events. Identifies timestamps. Applies field extraction rules from props.conf and transforms.conf. Usually happens on the Indexer.

Stage 3 — INDEXING

Parsed events are written to disk in an Index. Splunk writes two things: compressed raw data and TSIDX index files. These storage directories are called Buckets.

Stage 4 — SEARCHING

User interaction with indexed data. SPL queries, dashboards, alerts, reports. The Search Head sends queries to indexers, receives results, merges and presents them.

⚠ **NOTE: Splunk does NOT look at the content of data during the Input stage. Content inspection only happens during Parsing. Frequently tested interview point.**

⚠ **NOTE: The Universal Forwarder is the ONLY Splunk component that is NOT a full Splunk Enterprise installation. Every other component is a full installation configured differently.**

Key Definitions

Term	Definition
Event	Single row of data; minimum fields: _time, host, source, sourcetype
Index	Directory on disk where events are stored after indexing
Bucket	Storage subdirectory inside an index; data ages through bucket states
Source Type	Label on data that tells Splunk what format it is in
SPL	Search Processing Language — Splunk's native query language

Splunk Enterprise vs Splunk Cloud Platform

Aspect	Splunk Enterprise	Splunk Cloud Platform
Managed by	You (your team)	Splunk (as a service)
CLI access	Full CLI access	No CLI without support ticket
Config files	Edit directly	Submit support ticket to Splunk
App install	Any app from Splunkbase	Only vetted/approved apps
Updates	You manage them	Splunk auto-applies them
Unique inputs	Standard	AWS Firehose, Edge Processor

Phase 1 Summary

Topic	Key Point
What is Splunk	Centralized log analysis and machine data platform
Core problem solved	Manual log review at scale is impossible
Data pipeline	Input → Parsing → Indexing → Searching
Input stage	Receives raw data, stamps metadata, does NOT read content
Parsing stage	Creates events, extracts timestamps and fields
Indexing stage	Writes compressed raw data + TSIDX files (buckets) to disk
Searching stage	User queries, dashboards, alerts, reports

Event	Single row of data; minimum fields: _time, host, source, sourcetype
Universal Forwarder	Only component NOT a full Splunk Enterprise install
Splunk Enterprise	Self-managed, full CLI, edit config files directly
Splunk Cloud	Splunk-managed, no CLI, no direct config file editing

PHASE 2 — CORE BUILDING BLOCKS

Splunk Components

Processing Components

Component	Category	Key Responsibility
Search Head	Processing	UI, search queries, merges results from indexers; does NOT store data
Indexer	Processing	Parse, index, store data in buckets; serves search results
Universal Forwarder	Processing	Lightweight agent; collect and ship data ONLY; not full install
Heavy Forwarder	Processing	Full install; can parse, transform, filter, route data

Management Components

Component	Category	Key Responsibility
Indexer Cluster Manager	Management	Manages indexer cluster; does NOT store data itself
Deployer	Management	Manages Search Head Cluster members via config bundles
Deployment Server	Management	Manages forwarders and components at scale (Agent Management)
License Manager	Management	Holds license; all licensed components register here
Monitoring Console	Management	Free health dashboard component; included with Enterprise

⚠ NOTE: The Deployer manages Search Head Clusters. The Deployment Server manages forwarders. These are different components with different jobs — a very common interview trap.

⚠ NOTE: Splunk supports clustering for Indexers and Search Heads only. For License Manager and Heavy Forwarder — active-passive failover only, no clustering.

Deployment Models

S1 — Single Server Deployment

Search Head and Indexer are the same machine. Used for non-critical data, small teams, up to 500 GB/day. No cluster manager needed.

C1 — Distributed Clustered, Single Site

Search and Indexing tiers are separated. Standalone search heads (not clustered). Indexer Cluster with data replication. Management tier includes: License Manager, Monitoring Console, Indexer Cluster Manager, Deployment Server.

C3 — Distributed Clustered with Search Head Clustering

Full enterprise architecture. Adds Search Head Cluster on top of C1. A Deployer manages SH cluster members. A Captain is elected from cluster members to schedule jobs. This is what large organizations use.

Architecture	Search Tier	Index Tier	Management Tier
S1	SH + Indexer (same machine)	Same as search	Monitoring Console (optional)
C1	Standalone Search Heads	Indexer Cluster	License Mgr, Cluster Mgr, Deployment Server
C3	Search Head Cluster + Captain	Indexer Cluster	Deployer + all C1 management

Indexes and Bucket Lifecycle

An Index is a repository — a directory on disk — where events are stored after being parsed. Splunk writes two things: compressed raw data and TSIDX index files. The default index is called main.

Bucket Lifecycle

Bucket	Read/Write	TSIDX?	Trigger to Roll
Hot	Read + Write	Yes	Restart, max hot buckets, time span
Warm	Read only	Yes	max_warm_db_count exceeded
Cold	Read only	Yes	Age or size policy
Frozen	Read only	NO	coldToFrozenDir configured; TSIDX removed
Thawed	Read only	Yes (rebuilt)	Manual: copy + splunk rebuild + restart

⚠ NOTE: When data moves to the Frozen bucket, the TSIDX files are REMOVED. Only compressed raw journal.gz remains. Frozen data is NOT directly searchable — must run splunk rebuild first.

⚠ NOTE: Hot bucket is the ONLY bucket that supports both reading AND writing. All other buckets are read-only.

⚠ **NOTE: Data is NOT deleted on Frozen unless you have NOT configured a frozendb path. Configure coldToFrozenDir in indexes.conf to archive instead of delete.**

Bucket Storage Paths on Disk

```
/opt/splunk/var/lib/splunk/<index_name>/
    db/          -->  hot and warm buckets
    colddb/      -->  cold buckets
    frozendb/    -->  frozen buckets (if coldToFrozenDir configured)
    thaweddb/    -->  thawed (restored) buckets
```

Replication Factor and Search Factor

These terms only apply in an Indexer Cluster (C1 or C3 architecture).

Setting	What It Means	Rule
Replication Factor (RF)	Total copies of each bucket across peers	Survive RF-1 peer failures
Search Factor (SF)	Searchable copies (with TSIDX)	SF must always be ≤ RF
Non-searchable copy	Raw data only, no TSIDX	Smaller on disk; not directly queryable

Phase 2 Summary

Topic	Key Point
Search Head	UI + query execution; sends queries to indexers; does NOT store data
Indexer	Parses, stores, serves data; stores in indexes (buckets)
Universal Forwarder	Lightweight agent; collect and forward ONLY; not full Splunk install
Heavy Forwarder	Full Splunk install; can parse, transform, route, forward
Indexer Cluster Manager	Coordinates peer indexers; does NOT store data itself
Deployer	Manages Search Head Cluster members via config bundles
Deployment Server	Manages forwarders and components at scale
S1	Single server; SH + Indexer same machine; ≤500GB/day; non-critical
C1	Separate search and index tiers; indexer cluster; standalone SHs
C3	Adds Search Head Cluster on top of C1; full enterprise HA
Replication Factor	Total copies across peers; survive RF-1 failures
Search Factor	Searchable copies with TSIDX; SF must be ≤ RF
Hot bucket	Only bucket with read+write; active indexing
Frozen bucket	TSIDX removed; raw data only; deleted unless frozendb configured

PHASE 3 — SYNTAX AND CONFIGURATION

Configuration Files (.conf Files)

Every single behaviour in Splunk is controlled by plain text configuration files with a .conf extension. They are built on Stanzas (section headers in square brackets) and Attributes (key=value pairs under a stanza).

inputs.conf — Defines data inputs

```
[monitor:///var/log/syslog]
index      = main
sourcetype = syslog
disabled   = false
```

`[monitor:///var/log/syslog]` → stanza: monitor this file path
`index = main` → send events to the index named main
`sourcetype = syslog` → label this data as syslog type
`disabled = false` → this input is active and running

indexes.conf — Defines indexes and retention

```
[azure_logs]
homePath   = $SPLUNK_DB/azure_logs/db
coldPath   = $SPLUNK_DB/azure_logs/colddb
thawedPath = $SPLUNK_DB/azure_logs/thaweddb
maxTotalDataSizeMB = 51200
frozenTimePeriodInSecs = 2592000
```

`[azure_logs]` → create index named azure_logs
`homePath = ...` → path for hot and warm buckets
`coldPath = ...` → path for cold buckets
`maxTotalDataSizeMB = 51200` → maximum index size: 50 GB
`frozenTimePeriodInSecs = 2592000` → freeze data older than 30 days

outputs.conf — Lives on forwarders; defines destination

```
[tcpout]
defaultGroup = splunk_indexers

[tcpout:splunk_indexers]
server = 10.0.0.5:9997, 10.0.0.6:9997
```

`[tcpout]` → configure TCP output (standard forwarding)
`defaultGroup = splunk_indexers` → use the group named splunk_indexers by default
`server = 10.0.0.5:9997, ...` → send data to these indexers on port 9997

props.conf — Controls parse-time behaviour per sourcetype

```
[linux_secure]
TIME_PREFIX     = \s
MAX_TIMESTAMP_LOOKAHEAD = 24
```

```
BREAK_ONLY_BEFORE = ^\w{3}\s+\d+\s+\d{2}:\d{2}:\d{2}
REPORT-fields   = linux_secure_extractions
```

[linux_secure] → apply these settings to sourcetype=linux_secure
TIME_PREFIX = \s → timestamp starts after a whitespace character
BREAK_ONLY_BEFORE = ... → start new event when line begins with this regex
REPORT-fields = linux_secure_extractions → apply extraction named linux_secure_extractions from transforms.conf

⚠ NOTE: SEDCMD in props.conf masks sensitive data at INDEX TIME — before it is written to disk. This is irreversible. The original sensitive value is NEVER stored in Splunk.

transforms.conf — Holds actual regex for field extractions

```
[linux_secure_extractions]
REGEX  = sshd.*Invalid user (?P<invalid_user>\S+) from
(?P<src_ip>\d+\.\d+\.\d+\.\d+)
FORMAT = invalid_user::$1 src_ip::$2
```

[linux_secure_extractions] → name of this extraction (referenced by props.conf REPORT-)
REGEX = sshd.*Invalid user ... → the regular expression pattern to match in raw event text
(?P<invalid_user>\S+) → named capture group: extract non-whitespace as field invalid_user
(?P<src_ip>\d+\.\d+\.\d+\.\d+) → named capture group: extract IP address pattern as field src_ip
FORMAT = invalid_user::$1 src_ip::$2 → map captured groups to field names

Configuration File Precedence

The same .conf file can exist in multiple locations. Splunk reads all of them and merges them. Priority order from highest to lowest:

- 1. $SPLUNK_HOME/etc/system/local/ — HIGHEST PRIORITY
- 2. $SPLUNK_HOME/etc/apps/<app>/local/
- 3. $SPLUNK_HOME/etc/apps/<app>/default/
- 4. $SPLUNK_HOME/etc/system/default/ — LOWEST PRIORITY

⚠ NOTE: NEVER edit files in the default directory. Default files are overwritten during Splunk upgrades. Always make your customizations in the local directory.

BTool — Debug Configuration Precedence

```
splunk cmd btool inputs list --debug
splunk cmd btool props list --debug
splunk cmd btool <confname> check
```

btool inputs list --debug → show all merged inputs.conf settings and which file each came from
btool props list --debug → show all merged props.conf settings and which file each came from
btool <confname> check → validate conf file for syntax errors; shows line number of invalid keys

Source Types

A Source Type is a label assigned to incoming data. It tells Splunk what format the data is in. All field extraction rules, event breaking rules, and timestamp parsing rules in props.conf are keyed to the source type.

Built-in Source Type	Data It Handles
access_combined	Apache / Nginx web server access logs
linux_secure	Linux SSH authentication logs (/var/log/secure)
syslog	Standard syslog format
mysql:error	MySQL error logs
WinEventLog	Windows Event Log data

⚠ NOTE: If the source type is wrong, field extractions will NOT fire. Data appears as raw unformatted text. The fix requires re-indexing the data with the correct source type.

Regex Meta Characters

Character	Meaning
\d	Any digit (0-9)
\w	Any word character (A-Z, a-z, 0-9, underscore)
\s	Any whitespace (space, tab)
\S	Any non-whitespace character
.	Any single character
*****	Zero or more of previous
+	One or more of previous
?	Zero or one (optional)
{n}	Exactly n repetitions
[abc]	Any one of a, b, or c
[^abc]	Anything except a, b, or c
^	Start of line
\.	Literal dot (escaped)
(?P<name>regex)	Named capture group — creates a Splunk field

Phase 3 Summary

Topic	Key Point
inputs.conf	Defines data inputs; monitor paths; sourcetype assignment
indexes.conf	Defines indexes; storage paths; max size; frozenTimePeriodInSecs
outputs.conf	Lives on forwarders; defines destination indexer IP:9997
props.conf	Parse-time behaviour per sourcetype; event breaking; timestamps; REPORT-; SEDCMD

transforms.conf	Holds actual regex patterns for field extractions
Precedence order	system/local > apps/local > apps/default > system/default
Never edit default	Default files overwritten on upgrade; always use local directory
BTool	CLI tool to debug which conf file wins and validate syntax
Source Type	Label on data; drives all parsing rules in props.conf
Wrong source type	Field extractions do not fire; data must be re-indexed to fix
REPORT-	Key in props.conf that links to a named extraction in transforms.conf
SEDCMD	Masks sensitive data at index time; irreversible; original never stored
Named capture group	(?P<field_name>regex) syntax in transforms.conf
IFX	GUI tool for field extraction; generates regex automatically

PHASE 4 — WORKFLOW

End-to-End Data Flow

This is the most commonly asked interview question: walk me through how data gets from a Linux server into a Splunk search result.

```
Azure VM                UF                      Indexer                 Search Head
/var/log/app.log  --->  inputs.conf  --->     Parse + Index  --->    SPL Query
                        outputs.conf           props.conf              Results
                        TCP port 9997          transforms.conf
                                               Writes to buckets
```

⚠ NOTE: The Universal Forwarder does NOT parse data. It only collects and ships. Parsing happens on the Indexer (or Heavy Forwarder in large deployments).

Important Port Numbers

Port	Purpose
9997	Forwarder to Indexer data receiving (TCP)
8000	Splunk Web UI access
8089	Splunk management/REST API port; also used by deploy-poll
8085	Indexer cluster replication port (peer to peer)
9900	Search Head cluster replication port (configurable)

Universal Forwarder Setup Workflow

Step 1 — Enable receiving on Indexer (do this FIRST)

Settings → Forwarding and Receiving → Configure Receiving → New Receiving Port → 9997

Step 2 — Install Universal Forwarder

```
wget -O splunkforwarder.deb 'https://download.splunk.com/...'
sudo dpkg -i splunkforwarder.deb
```

Step 3 — Create least-privileged user (security best practice)

```
sudo useradd -m splunk
sudo chown -R splunk:splunk /opt/splunkforwarder
```

`sudo useradd -m splunk` → create a user named splunk with home directory
`sudo chown -R splunk:splunk /opt/splunkforwarder` → make splunk user the owner of the forwarder directory

⚠ NOTE: NEVER run the Splunk Universal Forwarder as root. Always create a dedicated least-privileged user. This is a security best practice interviewers specifically ask about.

Step 4 — Start the forwarder

```
sudo /opt/splunkforwarder/bin/splunk start --accept-license
```

Step 5 — Add forward server (writes to outputs.conf)

```
sudo /opt/splunkforwarder/bin/splunk add forward-server 10.0.0.5:9997
```

`add forward-server` → configure a destination for forwarded data
`10.0.0.5:9997` → IP address of the indexer and the receiving port

Step 6 — Add monitor (writes to inputs.conf)

```
sudo /opt/splunkforwarder/bin/splunk add monitor /var/log
```

`add monitor` → configure a file or directory to watch
`/var/log` → monitor the entire /var/log directory and all files in it

Agent Management (Deployment Server)

The Deployment Server lets you centrally push configuration changes to thousands of Universal Forwarders at once without touching each server individually.

Two Key Concepts

- Deployment App: package of conf files (inputs.conf + outputs.conf) stored at /opt/splunk/etc/deployment-apps/ on Deployment Server
- Server Class: maps a group of forwarders (filtered by hostname/IP/OS) to a specific Deployment App

Connect forwarder to Deployment Server

```
sudo /opt/splunkforwarder/bin/splunk set deploy-poll 10.0.0.10:8089
```

`set deploy-poll` → tell this forwarder to check in with the Deployment Server
`10.0.0.10:8089` → IP address of the Deployment Server and management port 8089

⚠ **NOTE: Deployment Apps are stored on the Deployment Server at /opt/splunk/etc/deployment-apps/. When pushed, they arrive on the forwarder at /opt/splunkforwarder/etc/apps/**

SPL Searching Workflow

⚠ **NOTE: ALWAYS start your search with index= and sourcetype= filters. This dramatically reduces the amount of data Splunk has to scan. This is called search optimization.**

Key SPL Commands

Command	Example	What It Does
table	| table _time, clientip, status	Show only specified fields as columns
stats count by	| stats count by status	Count events grouped by field value
timechart	| timechart count by status	Count over time split by field; for trend charts
top	| top limit=5 src	Top 5 most common values with count and percentage
eval	| eval class=floor(status/100)*100	Create new calculated field from expression
iplocation	| iplocation src	Enrich IP with country, city, region
sort -field	| sort -count	Sort descending (- = descending, + = ascending)
dedup	| dedup src	Remove duplicate events based on field value
rename	| rename src as 'Source IP'	Rename field in output
geostats	| geostats count by Country	Geographic map-ready statistics

Boolean Operators in SPL

```
index=main failed password     -- implicit AND: must contain both words
index=main failed OR error     -- OR: contains either word
index=main failed NOT root     -- NOT: contains failed but NOT root
index=main status=404          -- field search: status field equals exactly 404
index=main status=4*           -- wildcard: status field starts with 4
```

⚠ **NOTE: AND is IMPLICIT in SPL. Writing 'failed password' is the same as 'failed AND password'. OR and NOT must always be written explicitly.**

Time in SPL

```
index=main earliest=-24h latest=now      -- last 24 hours
index=main earliest=-7d@d latest=@d      -- last 7 full days
index=main earliest=-30m                 -- last 30 minutes
```

Search Modes

Mode	When to Use	Performance
Smart (default)	General searching; Splunk auto-detects best results	Balanced
Fast	You know all your fields; performance critical	Fastest
Verbose	Unknown data; need maximum field discovery	Slowest

Reports and Alerts

Aspect	Report	Alert
Purpose	Deliver information on schedule	Notify when condition is met
Timing	Scheduled delivery	Triggered by condition
Output	Email PDF/CSV, dashboard panel	Email, Slack, webhook, script
Monitoring	Passive	Active / reactive
Use case	Weekly security summary email	Brute force attack detected

⚠ NOTE: If you have multiple reports scheduled, stagger their run times. Running 20 reports at exactly midnight creates a massive load spike on Splunk.

Dashboards and Panels

A Dashboard is a collection of Panels. Each Panel is powered by an SPL search. Dashboard Inputs make dashboards interactive using Tokens.

Dashboard Input Types

Input Type	Token Syntax in SPL	Use Case
Time Range Picker	Shared Time Picker token	Let users change time window for all panels
Dropdown	index=main status=$status_token$	Pre-populated list; user selects value
Text Box	index=main host=$host_token$	Free text input for hostname or IP

⚠ NOTE: Token syntax is $token_name$ — both dollar signs are REQUIRED. If you forget the dollar signs the input does nothing.

Phase 4 Summary

Topic	Key Point
End-to-end flow	Azure VM → UF collects → TCP 9997 → Indexer parses → Index → Search Head query

UF does NOT parse	Parsing happens on Indexer (or Heavy Forwarder)
Port 9997	Forwarder to Indexer data receiving
Port 8000	Splunk Web UI
Port 8089	Management/REST API; also used by deploy-poll
UF least-privileged user	Create dedicated splunk user; never run UF as root
add forward-server	Writes to outputs.conf; sets destination indexer
add monitor	Writes to inputs.conf; sets what data to collect
set deploy-poll	Connects forwarder to Deployment Server on port 8089
Deployment App	Package of conf files pushed from Deployment Server
Server Class	Maps forwarder group to Deployment App by hostname/IP filter
index= and sourcetype=	Always start search with these; search optimization
AND in SPL	Implicit; writing 'failed password' = 'failed AND password'
Report vs Alert	Report = passive scheduled delivery; Alert = active reactive trigger
Token syntax	$token_name$ in dashboard SPL; both dollar signs required

PHASE 5 — REAL WORLD STRUCTURE

Indexer Clustering — Configuration

Master Node Configuration (server.conf)

```
[clustering]
mode               = master
replication_factor = 2
search_factor      = 2
pass4SymmKey       = YourSecretKeyHere
cluster_label      = azure_prod_cluster
```

`mode = master` → this node is the cluster master/manager
`replication_factor = 2` → maintain 2 copies of every bucket across peers
`search_factor = 2` → both copies must be immediately searchable
`pass4SymmKey = ...` → shared authentication key; all peers must match this
`cluster_label = ...` → human-readable name for this cluster

Peer Node Configuration (server.conf)

```
[clustering]
mode          = peer
master_uri    = https://10.0.0.1:8089
pass4SymmKey  = YourSecretKeyHere
```

`mode = peer` → this node is a peer indexer
`master_uri = https://10.0.0.1:8089` → address of the master node; always port 8089
`pass4SymmKey = ...` → must match the key set on the master node exactly

Connect Search Head to Cluster

```
splunk edit cluster-config -mode searchhead -master_uri https://10.0.0.1:8089 -
secret YourKey
```

⚠ NOTE: NEVER make manual configuration changes directly on a peer indexer node in a clustered environment. Changes must go through the master node via configuration bundles.

Configuration Bundles — Pushing Changes to All Peers

Step	Details
Bundle location on master	/opt/splunk/etc/master-apps/
Peers receive bundle at	/opt/splunk/etc/slave-apps/
Push via GUI	Settings → Indexer Clustering → Edit → Push Configuration Bundle
Push via CLI	splunk apply cluster-bundle

Indexer Discovery (Recommended Forwarding Method)

Forwarder queries the master node for the current list of active peers dynamically. If a new peer is added, forwarders discover it automatically.

Master node (server.conf):

```
[indexer_discovery]
pass4SymmKey = YourSecretKeyHere
```

Forwarder (outputs.conf):

```
[indexer_discovery:master]
master_uri    = https://10.0.0.1:8089
pass4SymmKey  = YourSecretKeyHere

[tcpout:splunk_indexers]
indexerDiscovery = master
```

⚠ NOTE: Indexer Discovery is the RECOMMENDED approach for forwarding to an indexer cluster. Hard-coded peer IPs in outputs.conf is a maintenance nightmare at scale.

Search Head Clustering — Configuration

What Replicates Between SH Cluster Members

- Users and roles
- Saved searches and reports
- Alerts
- Dashboards
- Lookups, Event Types, Data Models, Macros

⚠ **NOTE: Apps are NOT replicated between search head cluster members automatically. Apps must be pushed via the Deployer. Installing an app directly on one member creates inconsistency.**

Step 1 — Configure Deployer (server.conf)

```
[shclustering]
pass4SymmKey  = YourSecretKeyHere
shcluster_label = azure_sh_cluster
```

Step 2 — Initialize each SH Cluster Member

```
splunk init shcluster-config \
  -mgmt_uri https://10.0.0.2:8089 \
  -replication_port 9900 \
  -shcluster_label azure_sh_cluster \
  -conf_deploy_fetch_url https://10.0.0.10:8089 \
  -secret YourSecretKeyHere
```

`-mgmt_uri https://10.0.0.2:8089` → management URI of THIS instance (its own IP)
`-replication_port 9900` → port used for replicating data between SH members
`-conf_deploy_fetch_url ...` → address of the Deployer
`-secret YourSecretKeyHere` → shared key matching the Deployer

Step 3 — Bootstrap the Captain (run on ONE member only)

```
splunk bootstrap shcluster-captain \
  -servers_list
'https://10.0.0.2:8089,https://10.0.0.3:8089,https://10.0.0.4:8089'
```

Step 4 — Push Apps via Deployer

```
# Place apps in:  /opt/splunk/etc/shcluster/apps/
splunk apply shcluster-bundle -target https://10.0.0.2:8089
```

Indexer Cluster vs Search Head Cluster Comparison

Aspect	Indexer Cluster	Search Head Cluster
Manager component	Cluster Manager (Master Node)	Deployer
Config push command	apply cluster-bundle	apply shcluster-bundle -target URI
Config bundle location	/etc/master-apps/	/etc/shcluster/apps/
Peers receive at	/etc/slave-apps/	/etc/apps/ (on each member)
Elected role	N/A	Captain (schedules jobs)
Min nodes	2 peers + 1 master	2 members + 1 deployer
Stores data	Yes	No (searches IDX cluster)

License Architecture

License Hierarchy

License Group (one per License Manager) → License Stack (stacked files; capabilities add up) → License Pool (allocated subset for specific indexers) → Indexers assigned to pool

License Type	Measured By	Notes
Volume-based	GB ingested per day (midnight to midnight)	Internal logs, duplicates, metadata do NOT count
Infrastructure-based	vCPU cores on search heads and indexers	Better for high-search, low-ingest patterns

⚠ NOTE: 45 warnings in any rolling 60-day window on a <100 GB/day stack = license violation. Stack becomes unsearchable. Contact Splunk to reset.

⚠ NOTE: One indexer can only be assigned to ONE license pool. Universal Forwarders have their own free license — do NOT assign them to pools.

Access Control

Access control in Splunk is built on three layers: Users → Roles → Capabilities. A Role is a named collection of capabilities and index permissions.

Built-in Role	Capabilities
admin	Full access to everything; manage users, roles, configs
power	Create and share knowledge objects; run searches
user	Basic search and dashboard viewing; personal knowledge objects
can_delete	Special role allowing deletion of indexed data

⚠ NOTE: Never modify built-in roles. Create custom roles that INHERIT from built-in roles and add only the additional capabilities needed.

⚠ NOTE: Role changes take effect at the NEXT login. If you modify a role while a user is logged in, they will not see the change until they log out and back in.

Splunk Directory Structure

Directory	Contents
$SPLUNK_HOME/bin/	Splunk executables including the splunk binary
$SPLUNK_HOME/etc/system/default/	Factory default conf files; NEVER edit these
$SPLUNK_HOME/etc/system/local/	Your global customizations; HIGHEST priority
$SPLUNK_HOME/etc/apps/<app>/local/	App local conf files; edit these
$SPLUNK_HOME/etc/deployment-apps/	Deployment apps pushed to forwarders
$SPLUNK_HOME/etc/master-apps/	Config bundles pushed to IDX cluster peers
$SPLUNK_HOME/etc/shcluster/apps/	Config bundles pushed to SH cluster members
$SPLUNK_HOME/var/lib/splunk/	All indexed data (hot, warm, cold buckets)

$SPLUNK_HOME/var/log/splunk/splunkd.log	PRIMARY TROUBLESHOOTING LOG

⚠ NOTE: splunkd.log is your first stop for any troubleshooting. When a forwarder is not sending data, when Splunk fails to start — splunkd.log will tell you exactly what went wrong.

Phase 5 Summary

Topic	Key Point
Master Node	Exactly one per cluster; coordinates peers; does NOT store data
pass4SymmKey	Shared secret; must match on ALL cluster nodes
Config bundle (IDX)	/etc/master-apps/ on master; pushed via apply cluster-bundle
Never edit peer directly	All changes must go via master config bundle
Indexer Discovery	Forwarder queries master for peer list dynamically; recommended
SH Captain	Elected member; schedules jobs; coordinates replication
Deployer	Pushes apps to SH cluster via apply shcluster-bundle
Apps do NOT replicate	Must be pushed via Deployer; never install directly on one member
License Group	One per License Manager; contains all licenses
License Pool	Allocated subset of stack; one indexer = one pool only
45 warnings in 60 days	Violation for <100GB/day stacks; stack becomes unsearchable
Users → Roles → Capabilities	Three-layer access control model
Never modify built-in roles	Inherit from them in custom roles instead
splunkd.log	Primary troubleshooting log; first stop for any issue

PHASE 6 — INTERVIEW FAVOURITES

UF vs Heavy Forwarder

Aspect	Universal Forwarder	Heavy Forwarder
Full Splunk install	NO	YES
Parses data	NO	YES
Transforms data	NO	YES
Routes to different indexers	NO	YES
Resource usage	Very lightweight	Resource-heavy
License	Free forwarder license	Requires forwarder license
Use case	Standard agent on servers	Pre-processing, filtering, routing sensitive data

Deployer vs Deployment Server

Aspect	Deployer	Deployment Server
Manages	Search Head Cluster members	Forwarders and Splunk components
Config location	/etc/shcluster/apps/	/etc/deployment-apps/
Push command	apply shcluster-bundle	Automatic on forwarder check-in
Targets	Search Head cluster members	Deployment clients (forwarders)

Knowledge Objects

Field Extractions

Create searchable key-value pairs from raw event text. Happen at SEARCH TIME by default — you can add new field extractions to already-indexed data and they work immediately without re-indexing.

Event Types

Saved category of events. Save a search as an event type with a name, optional color and priority. Matching events are tagged with the eventtype field.

Lookups — Three Steps

- Step 1: Upload CSV file → Settings → Lookups → Lookup Table Files → New
- Step 2: Create Lookup Definition → Settings → Lookups → Lookup Definitions → New
- Step 3: Create Automatic Lookup → Settings → Lookups → Automatic Lookups → New

Tags

Assign human-readable names to specific field-value combinations. Multiple different field values can share the same tag. Search with: tag=production

Field Aliases

Alternate name for an existing field. Multiple fields can share the same alias, letting you search across data sources that use different field names for the same concept.

Data Models and CIM

Data Models

Structured hierarchy built on top of indexed data. Root Event (requires index= constraint) or Root Search. Powers the Pivot Tool for drag-and-drop visualization without SPL.

⚠ NOTE: Root Event datasets REQUIRE an index constraint — must specify at least index=something. Without this Splunk scans all indexes which is extremely expensive.

CIM — Common Information Model

An add-on from Splunk containing pre-built Data Models with standardized field names. Purpose: normalize diverse data sources so they use the same field names for cross-source searching.

CIM Compliance Process

Step	Action
1. IDENTIFY	Find the CIM data model relevant to your data (e.g. Authentication for SSH logs)
2. OBSERVE	Look at what tags and fields the CIM data model requires
3. APPLY	Create an event type for your data; tag it with required CIM tag
4. VERIFY	Use the Pivot Tool on the CIM data model to confirm data appears

⚠ NOTE: Tags are REQUIRED for CIM compliance. Creating field aliases and field extractions alone is NOT enough — you MUST also tag your event types with the required CIM tags.

Masking Sensitive Data

```
[payment_app]
SEDCMD-mask_cc     = s/\d{4}-\d{4}-\d{4}-\d{4}/XXXX-XXXX-XXXX-XXXX/g
SEDCMD-mask_ssn    = s/\d{3}-\d{2}-\d{4}/XXX-XX-XXXX/g
SEDCMD-mask_pass   = s/password=\S+/password=REDACTED/g
```

`SEDCMD-mask_cc =` → name this masking rule (SEDCMD- prefix required)
`s/` → sed substitution: find this pattern...
`\d{4}-\d{4}-\d{4}-\d{4}` → ...matching 4 digits dash 4 digits dash 4 digits dash 4 digits
`/XXXX-XXXX-XXXX-XXXX/` → ...and replace with this masked value
`/g` → apply globally: replace ALL occurrences in the event

⚠ NOTE: SEDCMD masking is IRREVERSIBLE. Once data is indexed with masking applied, the original values are gone forever. Happens on Indexer or Heavy Forwarder — NOT on Universal Forwarder.

Troubleshooting Scenarios

Forwarder not sending data to Splunk

- 1. Check splunkd.log: /opt/splunkforwarder/var/log/splunk/splunkd.log
- 2. Test connectivity: nc -z 10.0.0.5 9997
- 3. Verify forward server: splunk list forward-server
- 4. Verify monitors: splunk list monitor
- 5. Check forwarder status: splunk status
- 6. In Azure: check NSG rules — outbound TCP 9997 must be allowed from the VM
- 7. Verify receiving is enabled on indexer at port 9997

App missing on some Search Head Cluster members

The app was installed directly on one cluster member. Apps are NOT automatically replicated. Fix: place app in /opt/splunk/etc/shcluster/apps/ on the Deployer, then run:

```
splunk apply shcluster-bundle -target https://<any_sh_member>:8089
```

Indexer peer showing 'not fully replicated'

A peer node was down during replication. Once all peers return online, the Master Node automatically instructs peers to replicate missing copies back to the required Replication Factor.

Sensitive data found in Splunk events

Add SEDCMD rule to props.conf on the Indexer for the relevant sourcetype. Restart Splunk. Verify by uploading a test event containing a fake value and confirming it appears masked.

Key CLI Commands Reference

Command	Purpose
splunk start / stop / restart / status	Control Splunk service
splunk add monitor /var/log	Add file monitor (writes inputs.conf)
splunk add forward-server IP:9997	Add forward destination (writes outputs.conf)
splunk set deploy-poll IP:8089	Connect forwarder to Deployment Server
splunk list monitor	Show all monitored paths on this forwarder
splunk list forward-server	Show all configured forward destinations
splunk cmd btool inputs list --debug	Debug inputs.conf precedence
splunk cmd btool <conf> check	Validate conf file syntax
splunk apply cluster-bundle	Push config bundle from master to IDX peers
splunk apply shcluster-bundle -target URI	Push config bundle from Deployer to SH members
splunk bootstrap shcluster-captain -servers_list	Bootstrap SH cluster captain
splunk rebuild <thaweddb_path>	Rebuild TSIDX files for thawed bucket restore
splunk edit cluster-config -mode searchhead	Connect search head to indexer cluster

Phase 6 Summary

Topic	Key Point
UF vs Heavy Forwarder	UF = lightweight collect+ship; HF = full install parse/route/filter
Deployer vs Deployment Server	Deployer = SH cluster; Deployment Server = forwarders
Field extractions timing	Happen at SEARCH TIME; can add to already-indexed data without re-indexing
Lookups 3 steps	Upload CSV → Create definition → Create automatic lookup

Tags for CIM	Tags are REQUIRED; field aliases alone are not enough
Data Model root constraint	Root Event requires index= constraint; otherwise scans everything
SEDCMD	Index-time masking; irreversible; original value never stored
SEDCMD happens on	Indexer or Heavy Forwarder; NOT on Universal Forwarder
Troubleshoot forwarder	splunkd.log → nc port test → list forward-server → Azure NSG rules
App missing on SH members	Use Deployer; never install app directly on one cluster member
splunkd.log location	/opt/splunk/var/log/splunk/splunkd.log
Token syntax in dashboards	$token_name$ — both dollar signs required
AND in SPL	Implicit; never needs to be typed
OR and NOT in SPL	Must be typed explicitly